

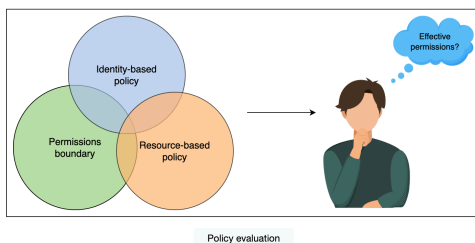
AWS IAM Permission Boundaries

Getting Started

When an identity-based policy is attached to an IAM entity, the permissions specified in the identity-based policy apply to that entity. However, a permissions boundary can also be attached to an IAM entity, and the permissions defined in it also apply to the entity. This makes things complex and makes it difficult to grasp which permissions will be applicable in such a case. Let's say that these permissions allow actions on a certain S3 bucket and that the S3 bucket also has a resource-based policy. This type of scenario makes it even more complex. Let's go through this lab to learn the policy evaluation logic.

Some of the terms that are frequently used throughout this lab are as follows:

- **IAM entities:** They're resource objects that IAM uses for authentication, i.e., users and roles.
- **Policies:** They're JSON documents that define permissions and access control rules for AWS resources. There are different types of policies. Some of them are as follows:
 - Identity-based policy
 - Permissions boundary
 - Resource-based policy

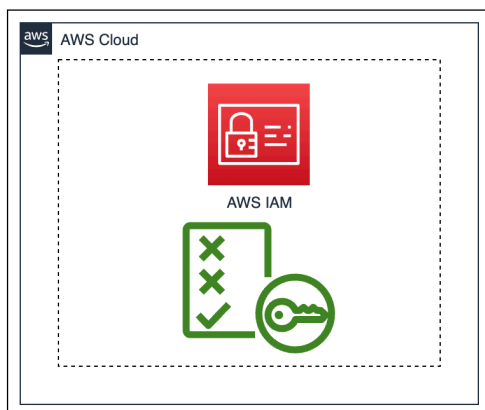


Create an IAM Identity-Based Policy

An **IAM policy** is a JSON document that allows us to define permissions for an IAM identity or a resource.

Once attached to an identity (user, group, or role) and termed an **identity-based policy**, the policy document allows us to define the actions or operations that the associated identity should be permitted (or denied) to do on the specified resources. Similarly, the policy document, once attached to a resource and termed as the **resource-based policy**, allows us to define the access permissions for the particular resource.

In this task, we'll focus on identity-based policies. After the completion of this task, the provisioned infrastructure will be similar to the one shown in the illustration below:



The architecture diagram

Note: To manage the permissions for this lab better, we have a `BasicS3Access` named pre-created policy at the start of the lab and because we already have this policy, you're not authorized to create one.

For your understanding, a policy can be created through the policy section of the IAM dashboard. You can update its JSON document to define any permissions that are required.

Structure of an identity-based policy

This JSON document of the policy can comprise one or more statements where each statement consists of the following elements:

Element	Explanation
Effect	This specifies if the associated actions are allowed or denied. Its value can either be <code>Allow</code> or <code>Deny</code> .
Action	This contains a list of actions.
Resource	This contains resources on which the actions will be effective.
Condition	This contains additional specific restrictions for controlling the effectiveness of actions.

Here's the JSON document for the pre-created policy `BasicS3Access` that we'll use in this lab:

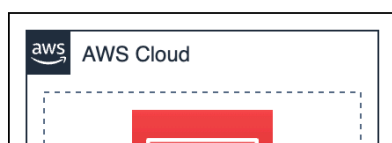
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "CreateDeleteBuckets",
      "Effect": "Allow",
      "Action": [
        "s3:CreateBucket",
        "s3:DeleteBucket",
        "s3:PutBucketPublicAccessBlock",
        "s3:GetBucketPolicy",
        "s3:PutBucketPolicy",
        "s3:PutEncryptionConfiguration"
      ],
      "Resource": "arn:aws:s3:::test-bucket-*",
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": "us-east-1"
        }
      }
    },
    {
      "Sid": "RequiredListPermission",
      "Effect": "Allow",
      "Action": "s3:List*",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": "us-east-1"
        }
      }
    },
    {
      "Sid": "CreateDeleteObject",
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:DeleteObject"
      ],
      "Resource": [
        "arn:aws:s3:::test-bucket-*/Public_data.txt",
        "arn:aws:s3:::test-bucket-*/Operations_data.txt",
        "arn:aws:s3:::test-bucket-*/Management_data.txt"
      ],
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": "us-east-1"
        }
      }
    }
  ]
}
```

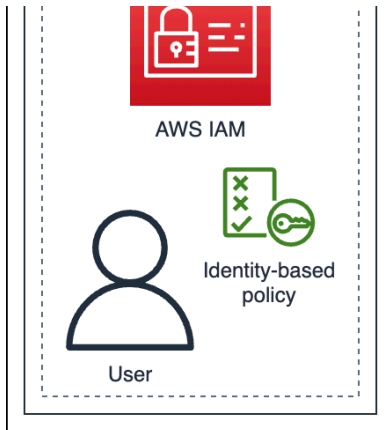
Next, let's create a user and attach the pre-created policy to provide the required permissions to that user.

Create an IAM User

IAM users are entities that represent individual users or applications that can access AWS services and resources. IAM users can have associated security credentials such as access keys and passwords. They can also have identity-based policies to manage authorization by specifying the actions, resources, and services a user is allowed to avail.

In this task, we'll create an IAM user and log in with it. After the completion of this task, the provisioned infrastructure will be similar to the one shown in the illustration below:





The architecture diagram

Set up an IAM user

Now, let's create an IAM user by following the given steps:

- Type "IAM" in the search bar on the AWS Management Console and click "IAM" from the search results. This takes us to the IAM dashboard.
- Click "Users" under "Access management" from the left sidebar. This takes us to a list of all available users in our account.
- Click the "Create user" button to create a new user.
- Write `LabUser` in the "User name" field.
- Check the "Provide user access to the AWS Management Console - *optional*" option.
 - For the "Console password" field, select the "Custom password" option and type `A123456a` as the password.
 - Uncheck the "Users must create a new password at next sign-in - Recommended" option and click the "Next" button.
- On the "Set Permissions" page, select the "Attach policies directly" option.
- Search and select the "BasicS3Access" policy and click the "Next" button.
- On the "Review and create" page, click the "Create user" button.
- We'll now see the "Console sign-in details" section. Copy the URL in the "Console sign-in URL" field, the user name in the "User name" field, and the password in the "Console password" field, and keep them safe or download the credentials by clicking the "Download .csv file" button.
- Click the "Return to users list" button.

Log in as the new user

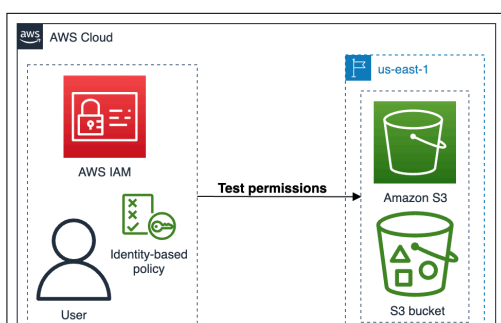
Now, let's log in as the new user we've just created. Follow the given steps to proceed:

- Open a new private window and use that to log in as the `LabUser` user without logging out from your current AWS session. We're using a private window because we can't have multiple AWS sessions running on the same browser.
- Navigate to the "Console sign-in URL" we copied in the previous step. This takes us to the AWS Management Console sign-in page with the account ID already entered.
- Enter `LabUser` in the "IAM user name" field.
- Enter the "Console password" that we copied in the last step in the "Password" field.
- Click the "Sign in" button to log in to the console.

We've successfully signed in as the `LabUser` user. We'll set up the rest of the infrastructure using this newly created user.

Effective Permissions

In this task, we'll check the effective permissions imposed by the identity-based policy we attached to the user. After the completion of this task, the provisioned infrastructure will be similar to the one shown in the illustration below:



permissions for the `LabUser` user, it'll only be able to perform some basic operations on the S3 bucket.

Test the Amazon S3 permissions

Let's try out the Amazon S3 service permissions:

Note: Ensure that you're working within the `us-east-1` region because access to resources outside of this region has been restricted for this lab.

- On the "LabUser" AWS Management Console, type "S3" in the search bar and click "S3" from the search results. This takes us to the S3 dashboard.
- Click the "Create bucket" button.
- For the "Bucket type" field, select the "General purpose" option.
- Write `test-bucket-<RandomText>` in the "Bucket name" field. You can try giving it a different name. However, you won't be able to do so because the policy attached to `LabUser` doesn't allow it.

Note: The S3 bucket name should be globally unique because a bucket can be used as a publicly accessible URL. So, you can replace `<RandomText>` in the bucket name with any random text that makes the name unique.

- In the "Block Public Access settings for this bucket" section, make sure the "Block all public access" option is checked.
- Scroll down to the end of the page and click the "Create bucket" button. We'll now be redirected to the S3 Buckets page.

Upload objects in the S3 bucket

We've successfully created an S3 bucket. Now, we'll upload a file into it by following the given steps:

- Search for the bucket we just created using the search bar in the "Buckets" section.
- Click the bucket name. We'll be redirected to the bucket page.
- Click the "Objects" tab.
- Click the "Upload" button in the "Objects" section. We'll be redirected to the upload page.
- Click the download icon on the top right of the widget below to download the files and unzip the downloaded file.

Note: We'll use these four files to check effective permissions in this later tasks as well.

- Click the "Add files" button in the "Files and folders" section.
- Select the "Confidential_data.txt" and "Management_data.txt" files along with the "Operations_data.txt" and "Public_data.txt" files to upload and click the "Upload" button at the end of the page. All the files will be uploaded except the "Confidential_data.txt" file because it's not allowed in the policy attached to the "LabUser" user.
- After the files are successfully uploaded, click the "Close" button.

Delete S3 bucket objects

With that, we've successfully uploaded the files to our S3 bucket. Now, let's delete them.

- Select all three files and click the "Delete" button.
- On the "Delete objects" page, type `delete` in the "Delete objects?" section and click the "Delete objects" button.
- After the files are successfully deleted, we'll see a success message. Click the "Close" button.

We were able to perform all the permissions listed in the identity-based policy. To check if you can perform actions not explicitly allowed in the policy, try uploading some other files to the S3 bucket you created or create a bucket without `test-bucket-` as a prefix in its name. In conclusion, we won't be allowed to perform any actions not allowed in the identity-based policy.

Introduction to Permissions Boundary

In Amazon Web Services (AWS), permissions boundaries are a feature that allows us to control the maximum permissions that an IAM entity (such as a user, group, or role) can have. They act as a way to set an upper limit on the permissions that can be granted to IAM entities, helping to prevent excessive or unintended access to AWS resources.

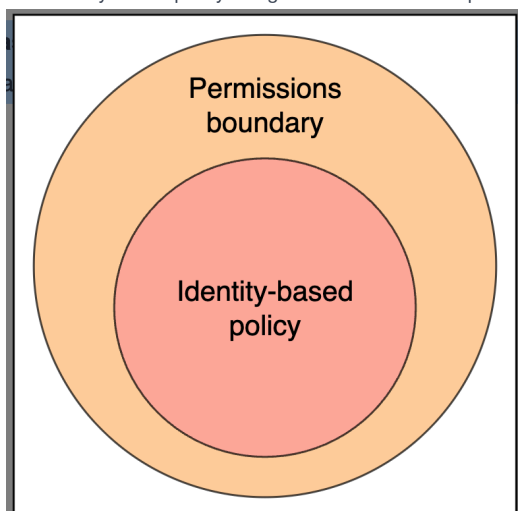
- **Why prevent excessive access?** Permissions boundaries are typically used when you need to restrict access of an IAM entity. Different teams or individuals can have their own permissions boundaries, ensuring that they only have access to the resources necessary for their specific roles and responsibilities. For example, we might want to allow a group of users to have broad access to resources but restrict their access to certain sensitive resources.
- **How to prevent access?** Attach permissions boundaries that don't provide access to sensitive resources with the users who shouldn't have access to them.

Permissions boundary with identity-based policy

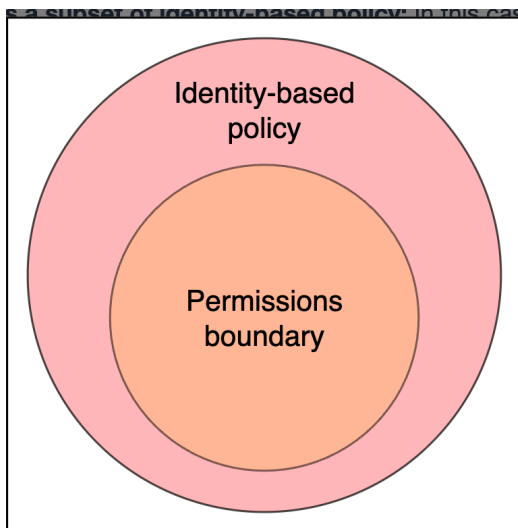
Permissions boundary, when attached to an IAM entity that already has an identity-based policy attached to it, can create the following scenarios:

- **Permissions boundary as a superset of identity-based policy:** In this case, the permissions boundary would allow all the actions that are allowed in the

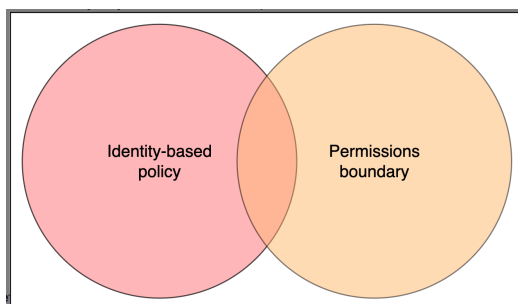
identity-based policy along with some additional permissions.



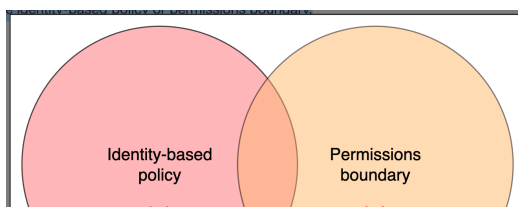
Permissions boundary as a subset of identity-based policy: In this case, the identity-based policy would allow all the actions that are allowed in the permissions boundary along with some additional permissions.



Overlapping permissions boundary and identity-based policy: In this case, the identity-based policy and permissions boundary only have some common permissions between them.



Denial effect in permissions boundary or identity-based policy: In this case, there would be a denial effect in either the identity-based policy or permissions boundary.

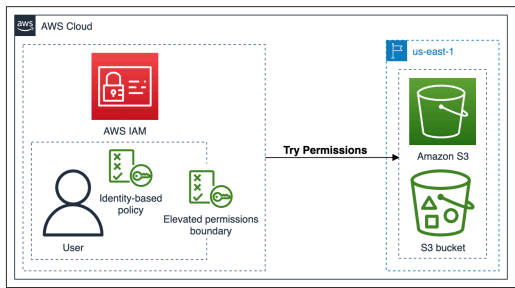




Now that we’ve discussed different cases that can arise while using permissions boundary with identity-based policy, let’s see which permissions will be effective in each case in the forthcoming tasks.

Permissions Boundary as a Superset of Identity-Based Policy

In this task, we’ll focus on the permissions boundary that would allow all the actions that are allowed in the identity-based policy, along with some additional permissions. After the completion of this task, the provisioned infrastructure will be similar to the one shown in the illustration below:



The architecture diagram

Policy for permissions boundary

We have a pre-created the `FullS3Access` policy. Here’s the JSON document for the pre-created policy `FullS3Access` that we’ll assign to the “LabUser” user as a permissions boundary:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllPermission",
      "Effect": "Allow",
      "Action": "s3:*",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": "us-east-1"
        }
      }
    }
  ]
}
```

Set permissions boundary

Follow the steps below to assign a permissions boundary to the “LabUser” user:

- Go back to the console of the main user with which we created the “LabUser” user, search “IAM” on the AWS Management Console, and click “IAM” from the search results. This takes us to the IAM dashboard.
- Click “Users” under “Access management” from the left sidebar. This takes us to a list of all available users in our account.
- Click the “LabUser” user and click the “Permissions boundary” section.
- Click the “Set permissions boundary” button.
- Search and select the “FullS3Access” policy and click the “Set boundary” button.

Let’s review both policies that are now attached to this user.

Element	Identity-Based Policy	Permissions Boundary
Effect	Allow	Allow
Action	Create and Delete bucket Put, Delete objects	All actions
Resource	test-bucket-... Public_data.txt, Operations_data.txt, Management_data.txt	All resources

According to identity-based policy, we can only create and delete buckets that have `test-bucket-` as a prefix in their name, and we can upload and delete three files, i.e., the “Public_data.txt” file, the “Operations_data.txt” file and the “Management_data.txt” file. Alternatively, the permissions boundary allows all actions in the AWS S3 service.

Try the permissions

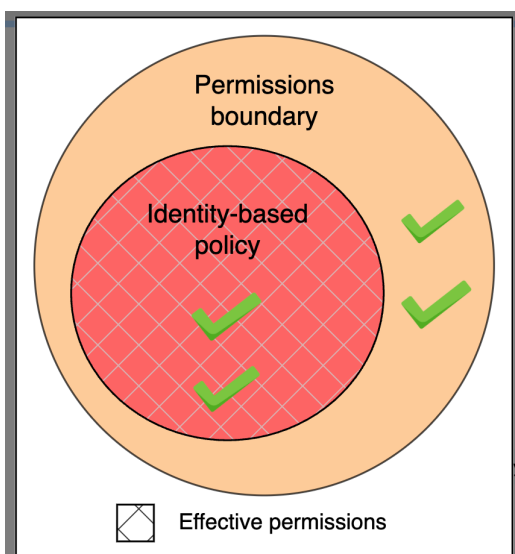
Now that we have the identity-based policy and elevated permissions boundary added to the user, let’s try the actions allowed in both types of permissions to

see which ones are effective.

- Navigate back to the “LabUser” AWS Management Console in incognito mode and create an S3 bucket with `test-bucket-` as a prefix in its name. You’ll be able to create it because both permissions allow it.
If you’re unsure about how to do this, click the “Hint: How to create an S3 bucket” button.
- Try to create a bucket that doesn’t have `test-bucket-` as a prefix in its name. Although the permissions boundary allows it, you won’t be able to create one.
- Navigate to any of the `test-bucket-<RandomText>` buckets we’ve created and try to add the four files again. You’ll be able to upload all files except the “Confidential_data.txt” file, even though the permissions boundary allows it.
If you’re unsure about how to do this, click the “Hint: How to add files to an S3 bucket” button.
- Click the “Close” button.
- Delete all the files in the bucket.
If you’re unsure about how to do this, click the “Hint: How to delete files from an S3 bucket” button.

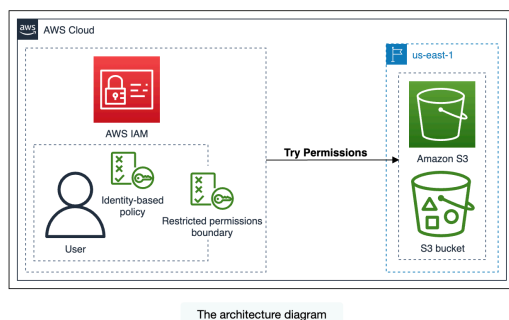
Conclusion

This brings us to the conclusion that only the permissions that are common in both the identity-based policy and permissions boundary are effective. Any additional permissions in the permissions boundary are ineffective.



Permissions Boundary as a Subset of Identity-Based Policy

In this task, we’ll focus on the permissions boundary that’ll only allow some of the permissions that are allowed in the identity-based policy. After the completion of this task, the provisioned infrastructure will be similar to the one shown in the illustration below:



Policy for permissions boundary

Permissions boundary has a structure similar to that of identity-based policy. We have a pre-created `RestrictedS3Access` named permissions boundary. Here’s the JSON document for the pre-created policy `RestrictedS3Access` that we’ll assign to the “LabUser” user as a permissions boundary:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "RequiredListPermission",
      "Effect": "Allow",
```

```

    "Action": "s3:List*",
    "Resource": "*",
    "Condition": {
      "StringEquals": {
        "aws:RequestedRegion": "us-east-1"
      }
    }
  },
  {
    "Sid": "CreateDeleteObject",
    "Effect": "Allow",
    "Action": [
      "s3:PutObject",
      "s3:DeleteObject"
    ],
    "Resource": [
      "arn:aws:s3:::test-bucket-*/Operations_data.txt",
      "arn:aws:s3:::test-bucket-*/Public_data.txt"
    ],
    "Condition": {
      "StringEquals": {
        "aws:RequestedRegion": "us-east-1"
      }
    }
  }
]
}

```

Change permissions boundary

Follow the steps below to assign a permissions boundary to the “LabUser” user:

- Go back to the console of the main user with which we created the “LabUser” user, search “IAM” on the AWS Management Console, and click “IAM” from the search results. This takes us to the IAM dashboard.
- Click “Users” under “Access management” from the left sidebar. This takes us to a list of all available users in our account.
- Click the “LabUser” user and click the “Permissions boundary” section to open it.
- Click the “Change boundary” button.
- Search and select the “RestrictedS3Access” policy and click the “Set boundary” button.

Let’s review both policies that are attached to the user:

Element	Identity-Based Policy	Permissions Boundary
Effect	Allow	Allow
Action	Create, Delete bucket Put, Delete objects	Put, Delete objects
Resource	test-bucket-... Public_data.txt, Operations_data.txt, Management_data.txt	Operations_data.txt, Public_data.txt

According to the identity-based policy, we can only create and delete buckets that have **test-bucket-** as a prefix in their name, and we can only upload and delete three files, i.e., the “Public_data.txt” file, the “Operations_data.txt” file, and the “Management_data.txt” file in the same bucket. Alternatively, the permissions boundary allows the upload and deletion of only two files, i.e., “Operations_data.txt” and “Public_data.txt” in the “test-bucket-...” S3 bucket.

Try the permissions

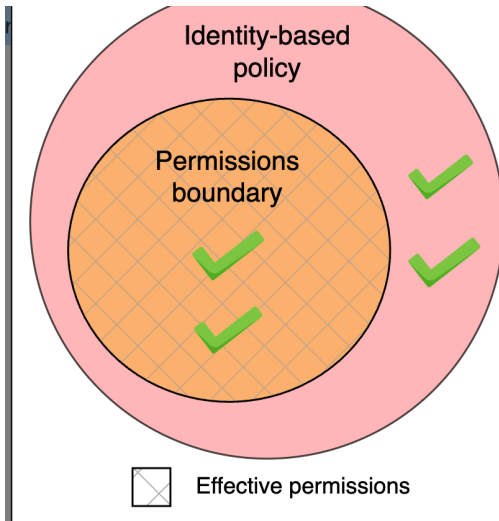
Now that we have both the identity-based policy and permissions boundary added to the user, let’s try the actions allowed in both types of permissions to see which ones are effective.

- Navigate back to the “LabUser” AWS console in incognito mode and create an S3 bucket with “test-bucket-” as a prefix in its name. You won’t be able to create it even though the identity-based policy allows it.
If you’re unsure about how to do this, click the “Hint: How to create an S3 bucket” button.
- Navigate to any of the “test-bucket-<RandomText>” buckets we’ve created and try to upload the files that are either permissible through identity-based policy or permissions boundary, i.e., the “Public_data.txt” file, the “Operations_data.txt” file, and the “Management_data.txt” file. You’ll be able to upload all files except the “Management_data.txt” file, even though the identity-based policy allows it.
If you’re unsure about how to do this, click the “Hint: How to add files to an S3 bucket” button.
- Click the “Close” button.
- Delete all the files in the bucket.
If you’re unsure about how to do this, click the “Hint: How to delete files from an S3 bucket” button.

Conclusion

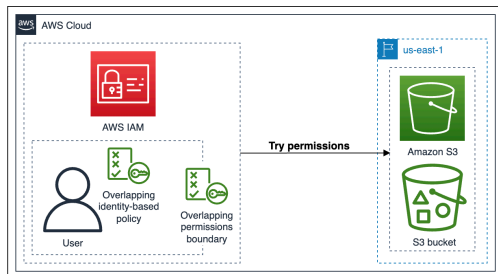
This solidifies our previous conclusion that only the permissions that are common in both the identity-based policy and permissions boundary are effective. Any additional permissions in the identity-based policy are ineffective.





Overlapping Permissions Boundary and Identity-Based Policy

In this task, we'll focus on the permissions boundary that would have some permissions common with the identity-based policy. After the completion of this task, the provisioned infrastructure would be similar to the one shown in the illustration below:



The architecture diagram

Policy for permissions boundary

Permissions boundary has a structure similar to that of identity-based policy. We have a pre-created `OverlappingS3Access` named permissions boundary. Here's the JSON document for the pre-created policy `OverlappingS3Access` that we'll assign to the "LabUser" user as a permissions boundary:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "RequiredListPermission",
      "Effect": "Allow",
      "Action": "s3:List*",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": "us-east-1"
        }
      }
    },
    {
      "Sid": "CreateDeleteObject",
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:DeleteObject"
      ],
      "Resource": [
        "arn:aws:s3:::test-bucket-*/Operations_data.txt",
        "arn:aws:s3:::test-bucket-*/Management_data.txt",
        "arn:aws:s3:::test-bucket-*/Confidential_data.txt"
      ],
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": "us-east-1"
        }
      }
    }
  ]
}
```

Let's review both policies that are attached to the user.

Element	Identity-Based policy	Permissions Boundary
Effect	Allow	Allow
Actions	Create, Delete bucket Put, Delete objects	Put, Delete objects
Resources	test-bucket-... Public_data.txt, Operations_data.txt, Management_data.txt	Confidential_data.txt, Management_data.txt Operations_data.txt

According to the identity-based policy, we can only create and delete buckets that have `test-bucket-` as a prefix in their name, and we can only upload and delete three files, i.e., the "Public_data.txt" file, the "Operations_data.txt" file, and the "Management_data.txt" file in the same bucket. Alternatively, the permissions boundary allows the upload and deletion of three files, i.e., the "Operations_data.txt" file, the "Management_data.txt" file, and the "Confidential_data.txt" file in the "test-bucket-..." S3 bucket.

Now that we've understood the overlapping permissions boundary, let's change the user's permissions boundary to the "OverlappingS3Access" policy. If you're unsure about how to do this, click the "Hint: How to change permissions boundary" button.

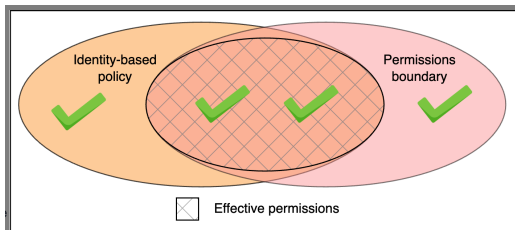
Try the permissions

Now that we have both the identity-based policy and permissions boundary added to the user, let's try the actions allowed in both types of permissions to see which ones are effective.

- Navigate back to the "LabUser" AWS Management Console in incognito mode and create an S3 bucket with `test-bucket-` as a prefix in its name. You won't be able to create it even though the identity-based policy allows it.
If you're unsure about how to do this, click the "Hint: How to create an S3 bucket" button.
- Navigate to any of the `test-bucket-<RandomText>` buckets we created previously and try to upload the files that are either permissible through identity-based policy or permissions boundary, i.e., the "Confidential_data.txt" file, the "Public_data.txt" file, the "Operations_data.txt" file, and the "Management_data.txt" file. You'll be able to upload all files except the "Confidential_data.txt" and "Public_data.txt" files, even though one of them is allowed by identity-based policy and the other by permissions boundary.
If you're unsure about how to do this, click the "Hint: How to add files to an S3 bucket" button.
- Click the "Close" button.
- Delete all the files in the bucket.
If you're unsure about how to do this, click the "How to delete the files in S3 bucket" button.

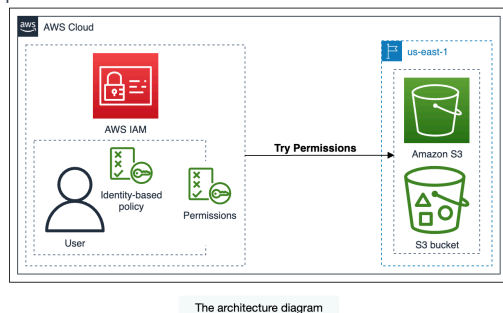
Conclusion

This further solidifies our previous conclusion that only the permissions that are common in both the identity-based policy and permissions boundary are effective. Any additional permissions in the identity-based policy or permissions boundary are ineffective.



Add a Denial Effect

In this task, we'll focus on the permissions boundary that'll have some actions explicitly denied on certain resources. After the completion of this task, the provisioned infrastructure would be similar to the one shown in the illustration below.



Policy for permissions boundary

Permissions boundary has a structure similar to that of identity-based policy. We have a `S3AccessWithDenial` named pre-created permissions boundary. Here's the JSON document for the pre-created policy `S3AccessWithDenial` that we'll assign to the "LabUser" user as a permissions boundary:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "RequiredListPermission",
      "Effect": "Allow",
      "Action": "s3:List*",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": "us-east-1"
        }
      }
    },
    {
      "Sid": "CreateDeleteObject",
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:DeleteObject"
      ],
      "Resource": [
        "arn:aws:s3:::test-bucket-*/Operations_data.txt",
        "arn:aws:s3:::test-bucket-*/Management_data.txt",
        "arn:aws:s3:::test-bucket-*/Confidential_data.txt"
      ],
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": "us-east-1"
        }
      }
    },
    {
      "Sid": "DontCreateDeleteObject",
      "Effect": "Deny",
      "Action": [
        "s3:PutObject",
        "s3:DeleteObject"
      ],
      "Resource": [
        "arn:aws:s3:::test-bucket-*/Management_data.txt"
      ],
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": "us-east-1"
        }
      }
    }
  ]
}

```

Let's review both policies that are attached to the user.

Element	Identity-Based Policy	Permissions Boundary	Permissions Boundary
Effect	Allow	Allow	Deny
Action	Create, Delete bucket Put, Delete objects	Put, Delete objects	Put, Delete objects
Resource	test-bucket-... Public_data.txt, Operations_data.txt, Management_data.txt	Confidential_data.txt, Management_data.txt, Operations_data.txt	Management_data.txt

According to identity-based policy, we can only create and delete buckets that have `test-bucket-` as a prefix in their name, and we can only upload and delete three files, i.e., the "Public_data.txt" file, the "Operations_data.txt" file, and the "Management_data.txt" file in the same bucket. Alternatively, the permissions boundary allows the upload and deletion of three files, i.e., the "Operations_data.txt" file, the "Management_data.txt" file, and the "Confidential_data.txt" file in the "test-bucket-..." S3 bucket. It also denies uploading and deleting the "Management_data.txt" file in the same bucket. Now that we have understood the permissions boundary with a denial effect, let's change the user's boundary permissions to the "S3AccessWithDenial" policy. If you're unsure about how to do this, click the "Hint: How to change permissions boundary" button.

Try the permissions

Now that we have both the identity-based policy and permissions boundary added to the user, let's try out the actions allowed in both types of permissions to see which ones are effective.

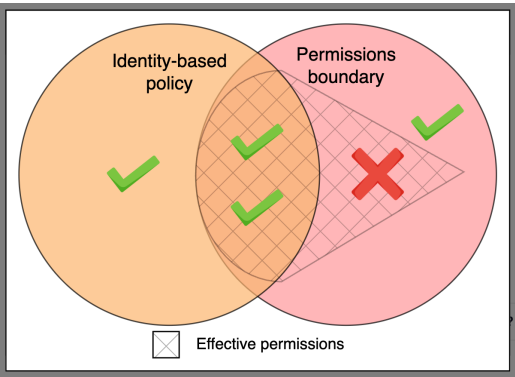
- Navigate back to the "LabUser" AWS Management Console in incognito mode and create an S3 bucket with `test-bucket-` as a prefix in its name. You won't be able to create it even though the identity-based policy allows it.
If you're unsure about how to do this, click the "Hint: How to create an S3 bucket" button.
- Navigate to any of the `test-bucket-<RandomText>` buckets we created previously and try to upload the files that are either mentioned in identity-based

policy or permissions boundary, i.e., the “Confidential_data.txt” file, the “Public_data.txt” file, the “Operations_data.txt” file, and the “Management_data.txt” file. You’ll only be able to upload the “Operations_data.txt” file. Even though “Management_data.txt” is allowed in both the identity-based policy and permissions boundary, upload operation still won’t be allowed because it has an explicit denial in the permissions boundary. If you’re unsure about how to do this, click the “Hint: How to add files to an S3 bucket” button.

- Click the “Close” button.
- Delete all the files in the bucket.
If you’re unsure about how to do this, click the “Hint: How to delete the files from an S3 bucket” button.

Conclusion

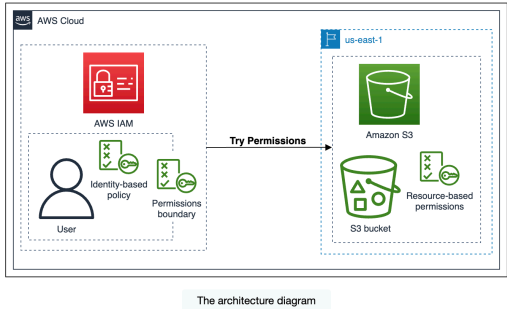
This brings us to the conclusion that the deny permissions defined in the permissions boundary always take precedence. However, other than that, only the common permissions in the identity-based policy and permissions boundary are effective.



Effective permissions with denial effect in permissions boundary

Use a Resource-Based Policy

Resource-based policies control access to AWS resources. We can attach these policies to AWS resources, such as S3 buckets, Lambda functions, and others, to specify who can access these resources and the actions they can perform on them. In this task, we’ll attach a resource-based policy to an S3 bucket. After the completion of this task, the provisioned infrastructure will be similar to the one shown in the illustration below:



Structure of resource-based policy

Similar to identity-based policies, resource-based policies have the same structure with one exception.

Element	Explanation
Effect	This specifies if the associated actions are allowed or denied. Its value can either be <code>Allow</code> or <code>Deny</code> .
Action	This contains a list of actions.
Resource	This contains resources on which the actions will effective.
Principal	This specifies the principal (such as an IAM user, roles, and others) who is allowed (or denied) access to the resource.
Condition	This contains additional specific restrictions for controlling the effectiveness of actions.

Here, we can see an extra `Principal` element for the resource-based policies. The identity-based policies don’t contain the `Principal` element because they’re attached to IAM identities separately.

Remove the permissions boundary

Follow the given steps to remove the permissions boundary from the user:

- In the console of the main user, search “IAM” and click “IAM” from the search results. This takes us to the IAM dashboard.
- Click “Users” under “Access management” from the left sidebar. This takes us to a list of all available users in our account.
- Click the “LabUser” user and click the “Permissions boundary” section to open it.
- Click the “Remove boundary” button and click the “Remove boundary” button on the confirmation pop-up as well.

Note: Without removing permissions boundary, we will not be able to add resource-based policy, because it is not an overlapping permission in identity-based policy and permissions boundary.

Add a resource-based policy

Follow the given steps to add a resource-based policy to the S3 bucket:

- In the console of the “LabUser” user, search “S3” and click “S3” from the search results. This takes us to the S3 dashboard.
- Click one of the buckets with the `test-bucket-` prefix in their names.
- Click the “Permissions” tab and scroll down to the “Bucket policy” section.
- Click the “Edit” button in the “Bucket policy” section.
- Copy the policy from the code widget below and paste it into the policy editor window.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::<account-id>:user/LabUser"
      },
      "Action": [
        "s3:PutObject",
        "s3:DeleteObject"
      ],
      "Resource": [
        "arn:aws:s3:::<bucket-name>/Public_data.txt",
        "arn:aws:s3:::<bucket-name>/Management_data.txt",
        "arn:aws:s3:::<bucket-name>/Confidential_data.txt"
      ],
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": "us-east-1"
        }
      }
    }
  ]
}
```

- Click the “Save changes” button.
- The permissions in the resource-based policy given above allow the “LabUser” user to perform the listed actions on three files, one of which isn’t included in the identity-based policy attached with the “LabUser” user, i.e., the “Confidential_data.txt” file. Let’s see the effectiveness of all three policies when used all at once in the next task.

Put It All Together

First, we have to set up the permissions boundary of the “LabUser” user to “S3AccessWithDenial” again.

If you’re unsure about how to do this, click the “Hint: How to set the permissions boundary” button.

We’ve attached the “BasicS3Access” identity-based policy and “S3AccessWithDenial” permissions boundaries to the “LabUser” user, and we’ve also attached a resource-based policy with the S3 bucket created with the “LabUser” user. We’ll only test the effective permissions for the S3 bucket in which we added the resource-based policy. Before testing the effective permissions, let’s review the permissions applicable in all three policies that will apply to the S3 bucket we’re working with.

Comparison of all three policies

Element	Identity-Based Policy	Permissions Boundary	Resource-Based Policy
Effect	Allow	Allow Deny	Allow
Action	Create, Delete bucket Put, Delete objects	Put, Delete objects	Put, Delete objects
Resource	test-bucket-... Public_data.txt, Operations_data.txt, Management_data.txt	Confidential_data.txt, Management_data.txt, Operations_data.txt	Public_data.txt, Management_data.txt, Confidential_data.txt

According to identity-based policy, we can only create and delete buckets that have `test-bucket-` as a prefix in their name, and we can only upload and delete three files, i.e., the “Public_data.txt” file, the “Operations_data.txt” file, and the “Management_data.txt” file in the same bucket. Alternatively, the permissions boundary allows the upload and deletion of three files, i.e., the “Operations_data.txt” file, the “Management_data.txt” file, and the “Confidential_data.txt” file in the “test-bucket-...” S3 bucket. It also denies uploading and deleting the “Management_data.txt” file in the same bucket. Moreover, the resource-based policy also allows the upload and deletion of three files, i.e., the “Public_data.txt” file, the “Management_data.txt” file, and the “Confidential_data.txt” file in the S3 bucket it’s attached to.

Try the permissions

Now that we have both the identity-based policy and permissions boundary added to the user, let's try the actions allowed in both types of permissions to see which ones are effective.

- Navigate back to the "LabUser" AWS Management Console in incognito mode and create an S3 bucket with `test-bucket-` as a prefix in its name. You won't be able to create it even though the identity-based policy allows it.

If you're unsure about how to do this, click the "Hint: How to create an S3 bucket" button.

- Navigate to any of the "test-bucket-`<RandomText>`" buckets we've created and try to upload the files that are either mentioned in identity-based policy or permissions boundary, i.e., the "Confidential_data.txt" file, the "Public_data.txt" file, the "Operations_data.txt" file, and the "Management_data.txt" file. You'll only be able to upload the "Operations_data.txt" file, the "Confidential_data.txt" file, and the "Public_data.txt" file. Even though "Management_data.txt" is allowed in the identity-based policy, the resource-based policy and the permissions boundary, upload operation still won't be allowed because it has an explicit denial in the permissions boundary.

If you're unsure about how to do this, click the "Hint: How to add files to an S3 bucket" button.

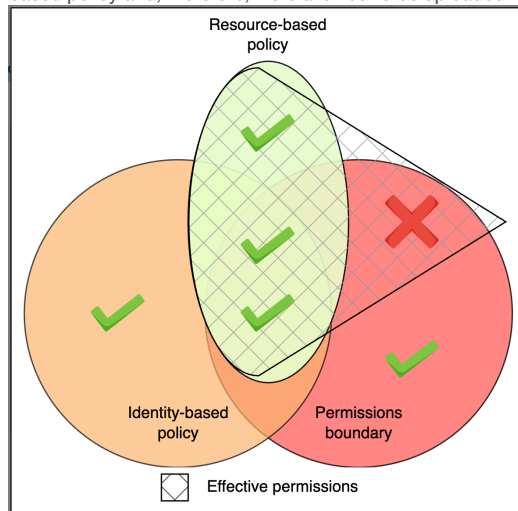
- Click the "Close" button.

- Delete all the files in the bucket.

If you're unsure about how to do this, click the "Hint: How to delete the files from an S3 bucket" button.

Conclusion

This brings us to the conclusion that the `Deny` action also takes precedence in this case. Apart from that, the common permissions in both the identity-based policy and permissions boundary are allowed, and all the permissions mentioned in the resource-based policy are allowed. In our example, "Confidential_data.txt" and "Public_data.txt" were not common in the identity-based policy and permissions boundary, but they're allowed in the resource-based policy and, therefore, were allowed to be uploaded.



To conclude, we have to remove the permissions boundary from the "LabUser" so that we can use the delete action of the identity-based policy to delete the buckets in the next task. If you're unsure about how to do this, click the "Hint: How to remove the permissions boundary" button.

Clean Up

Let's end this lab by cleaning up all the resources we've created in this lab. We'll use the same accounts to delete the resources that were used to create them.

Resources created by "LabUser"

We'll start with the resources created using the "LabUser" account.

Delete the S3 buckets

Let's start by deleting an S3 bucket:

- On the "LabUser" AWS Management Console, type "S3" in the search bar and click "S3" from the search results. This takes us to the S3 dashboard.
- Select a bucket that we created in this lab from the bucket list.
- Click the "Delete" button.
- Enter the bucket name in the textbox to confirm deletion and click the "Delete bucket" button.

Follow these steps to delete all the other S3 buckets we've created in this lab.

Resources created by the original account

Go back to the original browser window from where we started off and clean up the IAM user.

Delete the IAM user

Delete the IAM user

Follow the given steps to delete the IAM user:

- Head over to the “IAM > Users” page.
- Select “LabUser” from the list of users and click the “Delete” button.
- Enter “LabUser” in the confirmation pop-up and click the “Delete user” button to permanently delete the user.